

Web Services + Protocolo HTTP/JSON

Como sistemas diferentes conversam entre si pela internet

Conceito → do que é um Web Service ao protocolo HTTP • Prática → lendo contratos, testando com curl/Postman e manipulando JSON em Java

O que vamos ver hoje

- Parte 1 — Web Services: o que são, características e exemplos do dia a dia.
- Parte 2 — Protocolo HTTP: requisições, respostas, métodos, headers e códigos de status.
- Parte 3 — Formatos de dados e JSON: por que padronizar, sintaxe e uso em Java.
- Parte 4 — Arquiteturas de Web Services: panorama, frameworks e onde tudo isso roda.

1. Web Services

Conceitos e características

2. Protocolo HTTP

Requisição, resposta e status

3. Formatos & JSON

Padronizar dados entre sistemas

4. Arquiteturas

REST, SOAP, gRPC e mais

De onde viemos

- Já vimos que a Web funciona no modelo cliente-servidor: o cliente pede, o servidor responde.
- Estudamos a arquitetura da Web: navegador (cliente) e servidor trocando mensagens pela internet.
- Também vimos Orientação a Objetos em Java — a base para construirmos os sistemas por trás desses serviços.
- Hoje: como um servidor oferece funcionalidades para outros sistemas consumirem — os Web Services.

A Aula 06 conecta a teoria da Web (Aula 01) com a prática de construir e consumir serviços.

O que é um Web Service?

- É um serviço — uma funcionalidade — que pode ser acessado pela Web, usando a internet como meio de comunicação.
- Permite que sistemas diferentes, escritos em linguagens diferentes e rodando em plataformas diferentes, conversem entre si.
- Exemplo: um app mobile, um site em JavaScript e um sistema desktop em Java podem consumir o mesmo Web Service.
- Quem oferece o serviço é o servidor; quem usa o serviço é o cliente — o mesmo modelo cliente-servidor da Aula 01.

Cliente

App, site ou outro sistema que precisa de um dado ou de uma ação.

Web Service

Recebe a solicitação pela internet e executa a lógica do lado do servidor.

Resposta

O serviço devolve um resultado — dados, confirmação ou erro.

Características de um Web Service

- Interoperabilidade: consegue trabalhar com diferentes tipos de clientes, mesmo em plataformas distintas.
- Portabilidade: pode ser implantado em diferentes ambientes com impacto mínimo para quem já o consome.
- Escalabilidade: consegue atender múltiplos usuários ao mesmo tempo, crescendo conforme a demanda.
- Essas três características são o motivo de usarmos Web Services em vez de sistemas fechados e isolados.

Interoperabilidade

Java, PHP, .NET, mobile — todos conseguem consumir o mesmo serviço.

Portabilidade

Muda de servidor ou de nuvem sem quebrar quem já integra com ele.

Escalabilidade

De 10 a 10 mil usuários simultâneos, com os ajustes de infraestrutura corretos.

Web Service e Web API: qual a diferença?

- Web Service é o termo mais amplo: qualquer serviço acessível pela Web.
- Web API é o termo mais usado hoje em dia para descrever serviços que fornecem dados para outros sistemas.
- Uma API estabelece um padrão público de comunicação — rotas, formatos e regras — que diferentes clientes seguem para conversar com o servidor.
- Na prática, no mercado, os dois termos são usados quase como sinônimos. Vamos adotar 'API' a partir de agora.

Web Service

Termo mais amplo e mais antigo. Inclui SOAP, REST e outros modelos.

Web API

Termo mais atual. Foco em fornecer dados/recursos por um contrato público.

Web APIs que você já usa (mesmo sem saber)

- APIs famosas resolvem um problema específico e são consumidas por milhares de sistemas diferentes.
- Quando um app mostra um mapa, calcula frete ou processa um pagamento, ele quase sempre está chamando a API de outra empresa.
- Aprender a consumir e a construir APIs é o que nos permite integrar sistemas em vez de reinventar tudo do zero.

Google Maps

Mapas, geolocalização e rotas.

OpenWeather

Previsão do tempo em tempo real.

OpenAI

Modelos de linguagem (LLMs).

GitHub

Repositórios, issues e pull requests.

Stripe

Processamento de pagamentos.

PARTE 2

Protocolo HTTP

A linguagem que cliente e servidor usam para trocar mensagens pela Web

O que é HTTP?

- HTTP (HyperText Transfer Protocol) é o protocolo que define como cliente e servidor trocam mensagens na Web.
- Funciona no modelo requisição-resposta: o cliente envia uma requisição, o servidor devolve uma resposta.
- É stateless (sem estado): cada requisição é independente — o servidor não 'lembra' da requisição anterior por padrão.
- HTTPS é o HTTP com uma camada de criptografia (TLS) por cima, protegendo os dados durante o trajeto.

Requisição (Request)

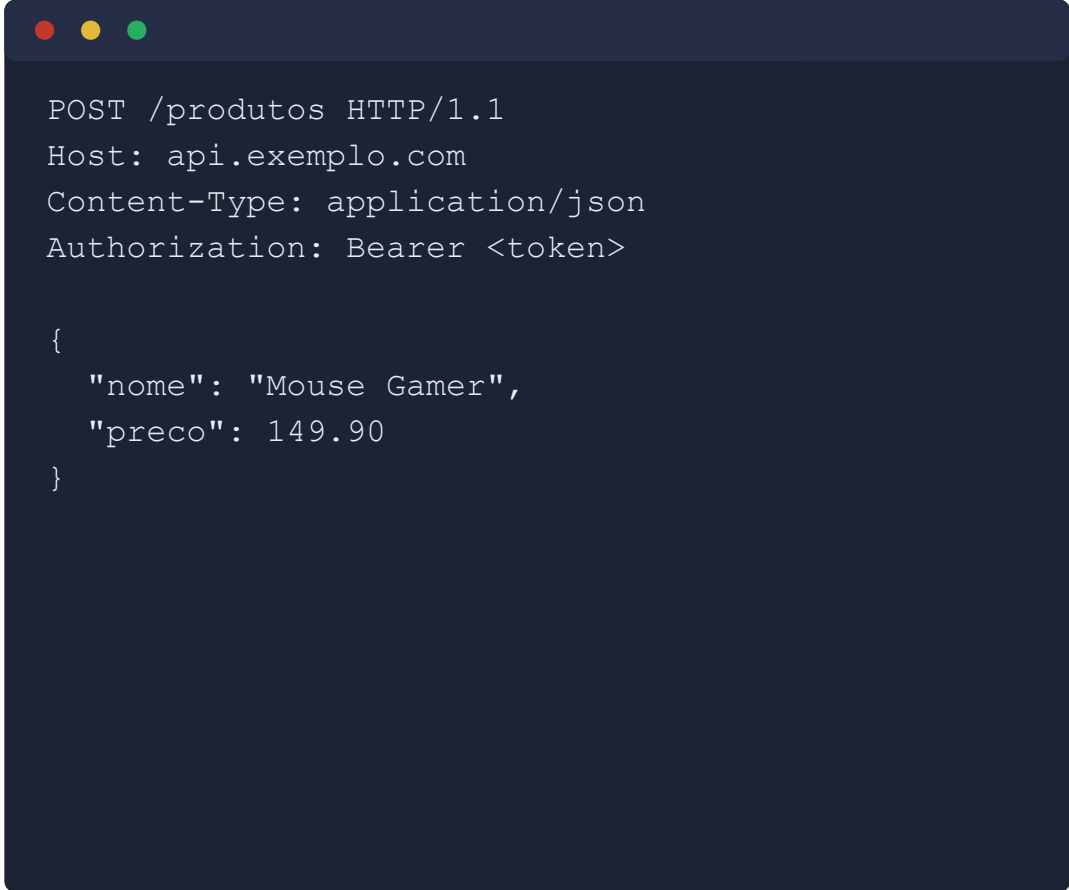
O cliente pede algo: um dado, uma ação, um recurso.

Resposta (Response)

O servidor devolve o resultado, com dados e status.

Anatomia de uma requisição HTTP

- Método: a ação desejada (GET, POST, PUT, DELETE...).
- Caminho (path): qual recurso está sendo acessado.
- Headers: metadados da requisição, como o formato dos dados enviados.
- Corpo (body): os dados enviados, quando necessário (ex.: em um POST).



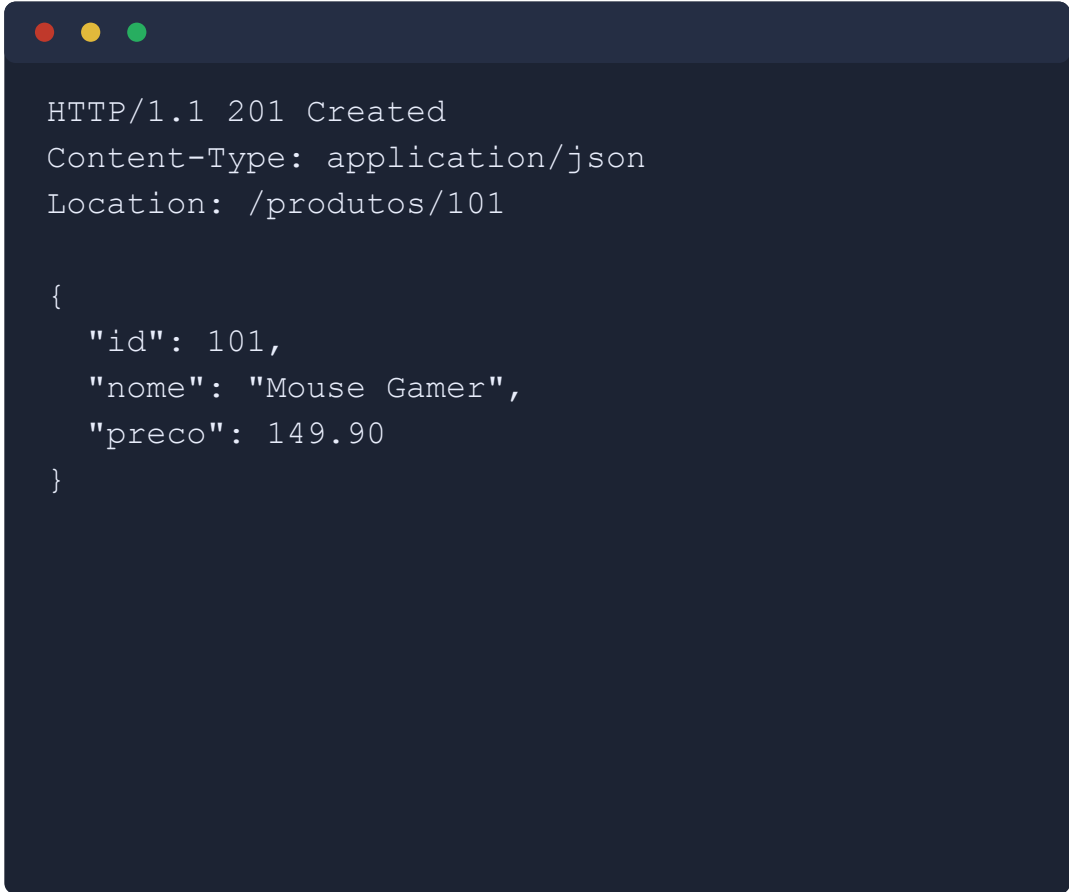
```
POST /produtos HTTP/1.1
Host: api.exemplo.com
Content-Type: application/json
Authorization: Bearer <token>

{
  "nome": "Mouse Gamer",
  "preco": 149.90
}
```

Essa requisição pede para criar (POST) um novo produto no recurso /produtos.

Anatomia de uma resposta HTTP

- Linha de status: versão do HTTP, código e mensagem (ex.: 201 Created).
- Headers: metadados da resposta, como o formato do corpo devolvido.
- Corpo (body): os dados de retorno, geralmente em JSON.
- O cliente usa o código de status para saber se deu tudo certo ou não, antes mesmo de ler o corpo.



```
HTTP/1.1 201 Created
Content-Type: application/json
Location: /produtos/101

{
  "id": 101,
  "nome": "Mouse Gamer",
  "preco": 149.90
}
```

201 Created confirma que o produto foi criado, e devolve o novo id gerado pelo servidor.

Métodos HTTP mais usados

Método	Para que serve	Envia corpo?	Exemplo
GET	Buscar/ler um recurso existente	Não	GET /produtos/101
POST	Criar um novo recurso	Sim	POST /produtos
PUT	Substituir um recurso inteiro	Sim	PUT /produtos/101
PATCH	Atualizar parte de um recurso	Sim	PATCH /produtos/101
DELETE	Remover um recurso existente	Geralmente não	DELETE /produtos/101

Idempotência: por que isso importa

- Um método é idempotente quando repetir a mesma requisição várias vezes tem o mesmo efeito de fazer uma única vez.
- GET, PUT e DELETE são idempotentes: chamar duas vezes não muda o resultado final.
- POST normalmente não é idempotente: enviar a mesma requisição duas vezes pode criar dois recursos.
- Isso importa na prática: se a internet cair e o cliente reenviar a requisição, o comportamento precisa ser previsível.

Idempotentes

GET, PUT, DELETE — repetir não muda o resultado final.

Não idempotente

POST — repetir pode criar registros duplicados.

Headers HTTP mais comuns

Header	Para que serve	Exemplo de valor
Content-Type	Formato dos dados enviados no corpo	application/json
Accept	Formato de resposta que o cliente aceita	application/json
Authorization	Credenciais de acesso do cliente	Bearer eyJhbGciOi...
User-Agent	Identifica o cliente que fez a chamada	PostmanRuntime/7.36
Cache-Control	Regras de cache da resposta	no-cache

Códigos de status HTTP: visão geral

- Todo resposta HTTP vem com um código de três dígitos que resume o resultado da requisição.
- Os códigos são agrupados em cinco categorias, pela primeira casa decimal.
- Aprender essas categorias já ajuda a entender uma resposta antes mesmo de olhar a documentação.

1xx — Informational

Requisição recebida, processo continua.

2xx — Sucesso

Tudo certo, requisição atendida.

3xx — Redirecionamento

É preciso ir a outro endereço.

4xx — Erro do cliente

O cliente pediu algo errado.

5xx — Erro do servidor

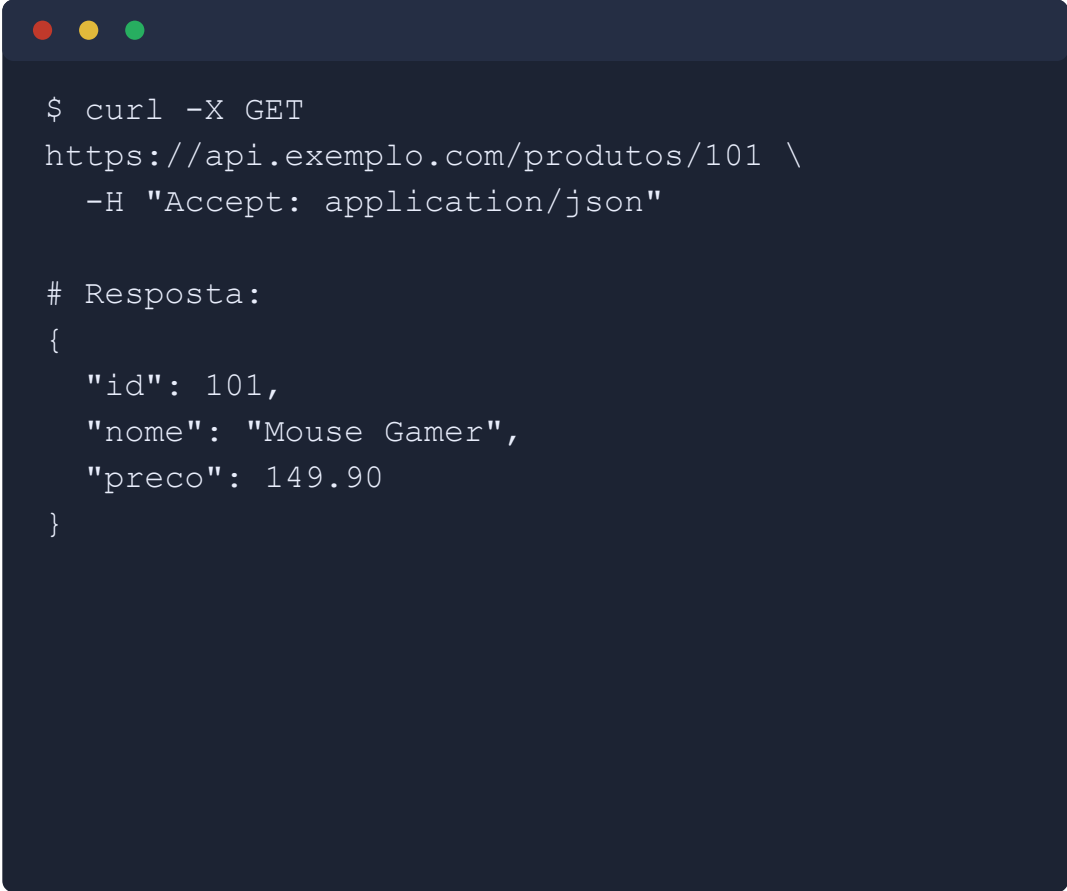
O servidor falhou ao processar.

Códigos de status mais usados no dia a dia

Código	Significado	Quando aparece
200 OK	Sucesso	Requisição atendida com sucesso (ex.: GET, PUT)
201 Created	Criado	Um novo recurso foi criado com sucesso (ex.: POST)
204 No Content	Sem conteúdo	Sucesso, mas sem corpo de resposta (ex.: DELETE)
400 Bad Request	Requisição inválida	Dados enviados pelo cliente estão incorretos
401 Unauthorized	Não autenticado	Faltam credenciais válidas de acesso
403 Forbidden	Sem permissão	O cliente é conhecido, mas não pode fazer isso
404 Not Found	Não encontrado	O recurso pedido não existe
500 Internal Server Error	Erro no servidor	Algo quebrou do lado do servidor

Fazendo uma requisição real com curl (GET)

- curl é uma ferramenta de linha de comando para enviar requisições HTTP — ótima para testar rápido, sem escrever código.
- -X define o método (aqui, GET é o padrão, então pode ser omitido).
- -H adiciona um header à requisição.
- A resposta vem no terminal, com o corpo em JSON.



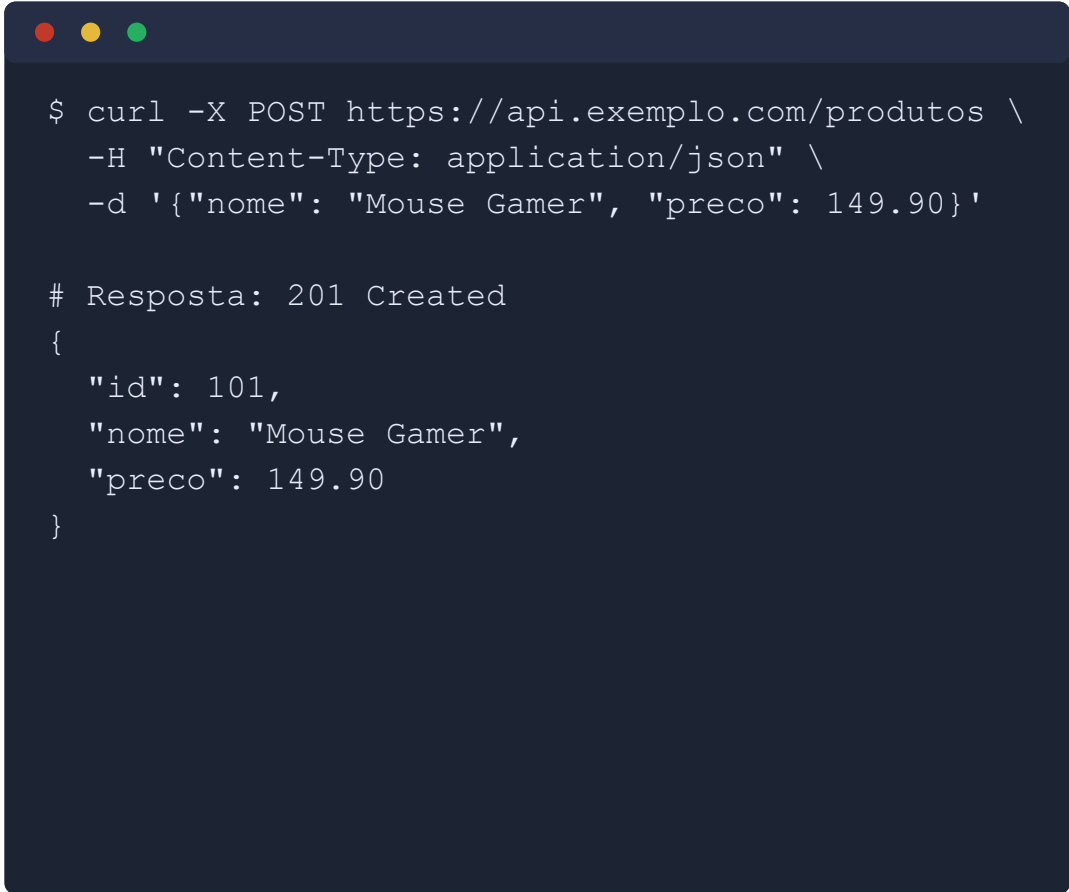
```
$ curl -X GET
https://api.exemplo.com/produtos/101 \
-H "Accept: application/json"

# Resposta:
{
  "id": 101,
  "nome": "Mouse Gamer",
  "preco": 149.90
}
```

Todo backend que você constrói deve poder ser testado assim, antes mesmo de existir uma tela.

Fazendo uma requisição real com curl (POST + JSON)

- -d envia o corpo (body) da requisição.
- É preciso declarar Content-Type: application/json para o servidor saber como interpretar o corpo.
- O servidor responde com 201 Created e o novo recurso, incluindo o id gerado.



```
$ curl -X POST https://api.exemplo.com/produtos \
-H "Content-Type: application/json" \
-d '{"nome": "Mouse Gamer", "preco": 149.90}'

# Resposta: 201 Created
{
  "id": 101,
  "nome": "Mouse Gamer",
  "preco": 149.90
}
```

Repare: mesma estrutura de sempre — método, rota, headers, corpo e status de resposta.

Testando APIs com Postman

- Postman é uma ferramenta gráfica para montar, enviar e organizar requisições HTTP, sem precisar de terminal.
- Permite salvar coleções de requisições — muito útil para documentar e testar uma API inteira.
- Environments guardam variáveis (como a URL base ou o token de autenticação) reutilizáveis entre requisições.
- Testar no Postman antes de programar o cliente evita perder tempo depurando dois lados ao mesmo tempo.

Requisições

Monta método, rota, headers e corpo visualmente.

Coleções

Agrupa e documenta as requisições de uma API.

Environments

Reaproveita URLs e tokens entre ambientes (dev, prod).

Contrato de API: o combinado entre cliente e servidor

- Um contrato de API define, de forma precisa, o que o cliente deve enviar e o que deve esperar de volta.
- Isso inclui: a rota, o método HTTP, o formato dos dados e o status esperado em cada cenário.
- Documentar esse contrato é o que permite que times diferentes construam cliente e servidor em paralelo, sem travar um no outro.

```
# Requisição
POST /produtos HTTP/1.1
Content-Type: application/json

{ "nome": "Mouse Gamer", "preco": 149.90 }

# Resposta esperada
HTTP/1.1 201 Created
Content-Type: application/json

{ "id": 101, "nome": "Mouse Gamer", "preco": 149.90 }
```

Esse contrato define rota, método, formato e status esperado — a base do que veremos em REST.

Interpretando um contrato de API

Objetivo: identificar método, rota, corpo esperado e status de sucesso a partir de um contrato dado.

- Leia o contrato ao lado com atenção.
- Identifique: qual é o método HTTP? Qual é a rota (path)?
- O que precisa ser enviado no corpo da requisição?
- Qual status de sucesso o servidor deve devolver, e o que vem no corpo da resposta?

```
# Contrato: cancelar um pedido

PUT /pedidos/{id}/cancelar HTTP/1.1
Content-Type: application/json

{ "motivo": "Cliente desistiu" }

# Resposta esperada
HTTP/1.1 200 OK

{ "id": 55, "status": "CANCELADO" }
```

PARTE 3

Formatos de Dados e JSON

Como cliente e servidor concordam sobre a 'forma' dos dados trocados

Por que precisamos de um formato padronizado?

- Web Services são acessados por plataformas diferentes: Java, JavaScript, Python, mobile, e por aí vai.
- Sem um formato combinado, cada sistema entenderia os dados de um jeito diferente — e a comunicação quebraria.
- Por isso, cliente e servidor combinam um formato de dados padronizado para enviar e receber informações.
- Hoje, o formato mais usado em APIs web é o JSON — mas vale conhecer as alternativas.

Formatos de dados mais comuns

Formato	Características	Onde é comum
JSON	Leve, legível, baseado em objetos e arrays	APIs REST modernas
XML	Verboso, baseado em tags, suporta esquemas rígidos	Integrações corporativas, SOAP
YAML	Legível por humanos, baseado em indentação	Arquivos de configuração
CSV	Tabular simples, separado por vírgulas	Planilhas e exportação de dados
Plain Text	Texto puro, sem estrutura definida	Logs e respostas simples

JSON: sintaxe básica

- JSON (JavaScript Object Notation) organiza dados em pares chave-valor, dentro de chaves { }.
- Os tipos básicos são: texto (string), número, booleano (true/false) e nulo (null).
- Chaves são sempre strings, entre aspas duplas.
- Vírgulas separam os pares — cuidado com a vírgula sobrando no último item, ela quebra o JSON.

```
{  
  "nome": "Mouse Gamer",  
  "preco": 149.90,  
  "disponivel": true,  
  "categoria": null  
}
```

JSON: objetos aninhados e arrays

- Um valor de JSON pode ser outro objeto — assim representamos relações entre dados.
- Um array (lista, entre colchetes []) guarda vários valores, inclusive vários objetos.
- Essa estrutura é o que permite representar, por exemplo, um pedido com vários produtos dentro.

```
{
  "id": 55,
  "cliente": { "nome": "Ana", "vip": true },
  "itens": [
    { "produto": "Mouse Gamer", "qtd": 1 },
    { "produto": "Teclado Mecânico", "qtd": 1 }
  ]
}
```

O mesmo dado, em JSON e em XML

JSON

```
{
  "produto": "Mouse Gamer",
  "preco": 149.90
}
```

XML

```
<produto>
  <nome>Mouse Gamer</nome>
  <preco>149.90</preco>
</produto>
```

JSON costuma ser mais enxuto e mais fácil de ler; XML é mais verboso, mas permite validação de esquema mais rígida.

Trabalhando com JSON em Java

- Em Java, a biblioteca Jackson é o padrão de mercado para converter entre objetos Java e JSON.
- `ObjectMapper.writeValueAsString()` transforma um objeto Java em uma String JSON (serialização).
- `ObjectMapper.readValue()` faz o caminho inverso: transforma um JSON em um objeto Java (desserialização).
- Frameworks como Spring Web já fazem essa conversão automaticamente nos controllers.

```
public class Produto {  
    private String nome;  
    private double preco;  
    // getters e setters  
}  
  
ObjectMapper mapper = new ObjectMapper();  
  
// Java -> JSON  
String json = mapper.writeValueAsString(produto);  
  
// JSON -> Java  
Produto p = mapper.readValue(json, Produto.class);
```

Convertendo um objeto Java em JSON

Objetivo: usar o Jackson (ObjectMapper) para serializar e desserializar a classe Produto.

- Crie a classe Produto com nome (String) e preco (double), com getters e setters.
- Crie um objeto Produto e converta-o em JSON com `writeValueAsString()`.
- Imprima o JSON gerado no console.
- Em seguida, converta esse mesmo JSON de volta em um objeto Produto com `readValue()`.

```
Produto produto = new Produto();
produto.setNome("Teclado Mecânico");
produto.setPreco(259.90);

ObjectMapper mapper = new ObjectMapper();
String json = mapper.writeValueAsString(produto);
System.out.println(json);

Produto convertido = mapper.readValue(json,
Produto.class);
```

PARTE 4

Arquiteturas de Web Services

Diferentes formas de estruturar a comunicação entre cliente e servidor

As principais arquiteturas de Web Services

Estilo	Formato típico	Melhor uso
SOAP	XML	Integrações corporativas e contratos formais
REST	JSON	APIs web tradicionais, CRUD de recursos
gRPC	Protobuf	Comunicação de alta performance entre microserviços
GraphQL	JSON	Consultas flexíveis para múltiplos tipos de cliente
WebSocket	Frames	Comunicação bidirecional em tempo real
Webhook	JSON	Notificações assíncronas orientadas a eventos

REST em poucas palavras

- REST organiza a API em torno de recursos (produtos, pedidos, clientes...), cada um com uma rota própria.
- Usa os verbos HTTP que já vimos (GET, POST, PUT, DELETE) para indicar a ação sobre o recurso.
- É stateless: cada requisição carrega tudo que o servidor precisa para entendê-la.
- É a arquitetura mais usada em APIs web hoje — vamos aprofundar REST com calma na próxima aula.

Comunicação em tempo real: WebSocket

- WebSocket não é uma arquitetura de API como REST ou SOAP — é um protocolo de comunicação.
- Ele mantém uma conexão bidirecional e persistente entre cliente e servidor.
- Diferente do HTTP tradicional, o servidor pode enviar dados para o cliente sem que ele precise pedir de novo.
- É a escolha certa quando dados precisam fluir continuamente, em vez de sob demanda.

Chats

Mensagens instantâneas entre usuários.

Jogos online

Sincronização de estado em tempo real.

Notificações

Alertas enviados assim que acontecem.

Dashboards ao vivo

Métricas atualizadas continuamente.

Comparando as abordagens

Tecnologia	Formato comum	Melhor uso
REST	JSON	CRUD e APIs web tradicionais
SOAP	XML	Integrações corporativas e contratos formais
gRPC	Protobuf	Comunicação entre serviços com alta performance
GraphQL	JSON	Consultas flexíveis e múltiplos clientes
WebSocket	Frames	Atualizações em tempo real

Frameworks para construir Web Services

- Frameworks cuidam da parte repetitiva (rotas, parsing de HTTP, conversão de JSON) para você focar na regra de negócio.
- No nosso curso, vamos construir APIs REST em Java usando o Spring Web.
- Vale conhecer as alternativas em outras linguagens — você vai encontrá-las no mercado.

Spring Web (Java)

O framework que usaremos no curso.

NestJS (TypeScript)

Inspirado em Angular, muito usado em Node.js.

Express (Node.js)

Minimalista e flexível.

Flask (Python)

Simples e direto para APIs pequenas.

Laravel (PHP)

Framework full-stack popular em PHP.

Spring Web: seu primeiro endpoint

- @RestController marca a classe como responsável por responder requisições HTTP.
- @GetMapping mapeia o método para responder requisições GET em uma rota.
- O valor retornado pelo método vira automaticamente o corpo da resposta.
- É este framework que vamos usar para construir a API REST do curso, a partir da próxima aula.

```
@RestController
public class HelloWorldController {

    @GetMapping("/")
    public String olaMundo() {
        return "Olá mundo";
    }
}
```

Microserviços e Monolitos

- Monolitos são sistemas completos, com múltiplos domínios de negócio, em uma única aplicação.
- Microserviços dividem o sistema em serviços menores e independentes, que trabalham em conjunto.
- Nenhuma abordagem é 'melhor' de forma absoluta — a escolha depende do tamanho e da complexidade do time e do produto.
- No curso, vamos começar com uma API única — o equivalente a um pequeno monolito — antes de falar em integração entre serviços.

Monolito

Um único sistema, com todos os domínios juntos. Mais simples de começar.

Microserviços

Vários sistemas menores, cada um com sua responsabilidade. Mais complexo, mais escalável.

Onde um Web Service roda?

- Para ficar disponível publicamente, um Web Service precisa estar hospedado em algum servidor, sempre ligado.
- Existem diferentes formas de infraestrutura para isso, cada uma com seu nível de controle e de complexidade.
- Vamos voltar a esse assunto com mais profundidade quando chegarmos no tópico de Infraestrutura, mais adiante no curso.

Servidor físico

Máquina própria, configurada e mantida manualmente.

VPS

Servidor virtual privado, contratado de um provedor.

Nuvem (Cloud)

AWS, Azure, Google Cloud — infraestrutura sob demanda.

Montando seu primeiro contrato de API

Objetivo: descrever, em texto, o contrato completo de uma API para gerenciar uma lista de tarefas.

- Defina a rota e o método para: criar uma tarefa, listar as tarefas e marcar uma tarefa como concluída.
- Para cada rota, escreva um exemplo de corpo de requisição em JSON (quando houver).
- Para cada rota, escreva um exemplo de corpo de resposta em JSON e o status HTTP esperado.
- Guarde este contrato: vamos implementá-lo de verdade nas próximas aulas, com Spring Web.

RECAPITULANDO

O que levamos para casa

- Web Services permitem que sistemas diferentes conversem, com interoperabilidade, portabilidade e escalabilidade.
- HTTP define o formato de requisição e resposta: método, headers, corpo e código de status.
- JSON é o formato padrão para trocar dados entre cliente e servidor — e o Jackson faz essa ponte em Java.
- Existem várias arquiteturas (REST, SOAP, gRPC, GraphQL, WebSocket); REST será o nosso foco daqui para frente.

Dever de casa

Finalize os 3 exercícios da aula (contrato de API, conversão Java/JSON e contrato da lista de tarefas) e traga suas dúvidas para a próxima aula, quando começamos REST na prática.

Para ir além

- MDN Web Docs — HTTP: referência completa sobre métodos, headers e status codes.
- json.org — a especificação oficial e minimalista do formato JSON.
- Documentação do Jackson (FasterXML) — todos os detalhes de serialização/desserialização em Java.
- Postman Learning Center — tutoriais oficiais para testar APIs na prática.

Repositório da disciplina: github.com/placidodaniel/backend-aulas-satc